

Version 2.0

January 2026



DATABASE

MODULE 2 - RELATIONAL DATA MODELING

Author:

Dr. Ir. Agus Eko Minarno, S.Kom., M.Kom., IPM.

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
RELATIONAL DATA MODELING.....	4
1. UNIQUE IDENTIFIER (UID).....	4
SIMPLE UID.....	5
COMPOSITE UID.....	5
ARTIFICIAL UID.....	6
2. RELATIONSHIP.....	6
CARDINALITY.....	7
OPTIONALITY.....	11
3. PRECISION AND SCALE.....	11
HOW TO USE PRECISION AND SCALE.....	12
EXAMPLE 1 (CORRECT USAGE).....	12
EXAMPLE 2 (CORRECT USAGE).....	13
EXAMPLE 3 (INCORRECT USAGE).....	13
EXAMPLE 4 (CORRECT USAGE).....	13
EXAMPLE 5 (INCORRECT USAGE).....	14
EXAMPLE 6 (CORRECT USAGE).....	14
PRACTICE ACTIVITY.....	15
TUTORIAL FOR ADDING RELATIONSHIPS TO A CONCEPTUAL DATA MODEL (CDM).....	15
TUTORIAL ON CONVERTING A CONCEPTUAL DATA MODEL (CDM) TO A PHYSICAL DATA MODEL (PDM).....	17
TUTORIAL FOR APPLYING PRECISION AND SCALE TO AN ATTRIBUTE.....	19
TRY OUT.....	19
TRY OUT INSTRUCTIONS.....	20
ASSIGNMENT.....	20
ASSIGNMENT REQUIREMENTS.....	20
ASSESSMENT.....	21
PRACTICUM GRADING RUBRIC.....	22



INTRODUCTION

OBJECTIVES

1. Students understand notation in database modeling (UID, Relationships).
2. Students understand the concept of relational data modeling.
3. Students are able to implement entity designs and inter-entity relationships.

MODULE TARGETS

1. Students can implement Unique Identifiers (UID) and Relationships correctly.
2. Students can apply relational data modeling concepts (1:1, 1:N, M:N) in their schema.
3. Students can convert a Logical (CDM) design into a Physical (PDM) design.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson.

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop or personal computer.
3. **Oracle SQL Developer Data Modeler** installed and ready to use.
[LINK-SQL Developer Data Modeler](#)
4. Strong motivation and commitment to learning.

KEYWORDS

Unique Identifier (UID), Relationship, Cardinality, Optionality, Transferability, PDM



MATERIAL

RELATIONAL DATA MODELING

A relational database is a type of database that stores data in the form of tables, where each table consists of rows and columns. Each row represents a single data record, while each column represents a specific attribute of that data. This structure allows data to be organized systematically and accessed efficiently.

In database design, a conceptual model is not sufficient for implementation. A conceptual model only describes what data exists and how data is conceptually related. To implement a database in a DBMS, the design must follow the rules of the relational model. This process is known as relational data modeling.

Relational data modeling focuses on ensuring that:

1. Every data record can be uniquely identified,
2. Relationships between entities are clearly defined,
3. Data integrity is preserved,
4. The structure is ready to be converted into physical tables.

1. UNIQUE IDENTIFIER (UID)

A Unique Identifier (UID) is an attribute, or a combination of attributes, that uniquely identifies each record within an entity. In relational data modeling, a UID plays a critical role because it ensures that every record can be distinguished from all other records.

An entity without a UID cannot be properly implemented in a relational database. This is because the DBMS requires a way to uniquely reference each record when performing operations such as data retrieval, updates, and relationship mapping.

A valid UID must satisfy the following conditions:

1. Each value must be unique for every record.
2. The value must never be null.
3. The value should remain stable and not change over time.

These rules ensure consistency and reliability in database operations.



SIMPLE UID

A Simple UID is a unique identifier that consists of only one attribute. This type of UID is used when a single attribute is sufficient to uniquely identify every record in an entity.

Simple UIDs are commonly found in entities where identification is straightforward, such as student IDs or employee numbers. Because the UID consists of only one attribute, the design is simpler and easier to maintain.

An example attribute that can function as a simple UID is `student_id`

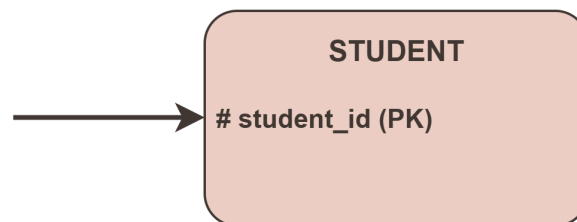


Figure 1. Simple UID

COMPOSITE UID

A Composite UID is a unique identifier formed by combining two or more attributes. Composite UIDs are used when no single attribute is able to uniquely identify a record by itself.

This situation often occurs in entities that represent relationships between other entities, such as intersection entities in many-to-many relationships. In such cases, the combination of multiple attributes is required to ensure uniqueness. For example, in a course enrollment entity, neither `student_id` nor `course_id` alone is sufficient to uniquely identify a record. However, the combination of both attributes can uniquely represent each enrollment.

Composite UIDs require careful design because they directly affect relationships, foreign keys, and future normalization processes.

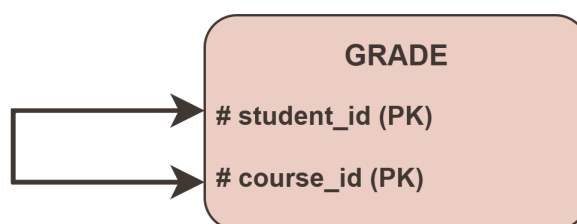


Figure 2. Composite UID



ARTIFICIAL UID

An Artificial UID is a unique identifier that is generated by the system and does not have inherent business meaning. Unlike natural attributes, artificial UIDs are created solely for identification purposes.

Artificial UIDs are often used when:

1. Natural identifiers are complex or too long,
2. Composite UIDs become difficult to manage,
3. System simplicity and performance are priorities.

Common examples of artificial UIDs include auto-increment numbers or generated codes. Although artificial UIDs simplify database design, they should be used carefully to ensure that business rules are still properly enforced.

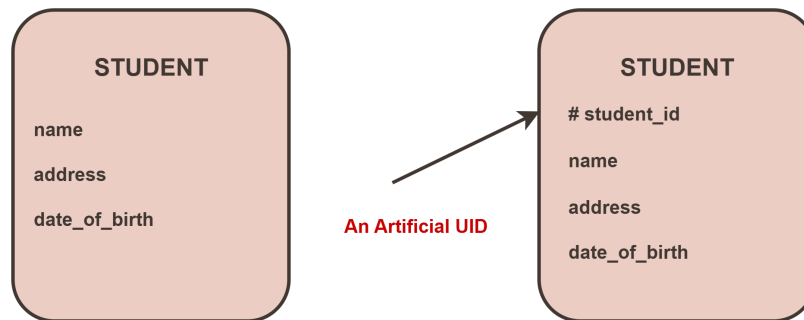


Figure 3. Example of Artificial UID

2. RELATIONSHIP

A relationship represents a meaningful association between two entities. Relationships describe how data in one table is connected to data in another table and reflect real-world interactions between objects or concepts.

In relational data modeling, relationships are not merely visual connections. They define how tables interact through foreign keys and determine how data can be queried, updated, and maintained.

Each relationship is defined by:

1. Its name,
2. Its cardinality,
3. Its optionality.



CARDINALITY

Cardinality defines how many instances of one entity can be associated with another entity in a database. **The correct cardinality is determined by real-world rules, not by personal interpretation or point of view.** To identify cardinality, designers must consider how the data behaves in reality and ask whether one instance of an entity can be related to one, many, or no instances of another entity. Cardinality must always be evaluated in both directions to avoid incorrect assumptions. When both entities can be associated with many instances of each other, the relationship is classified as many-to-many and must be handled carefully in relational design.

Example: Student and Course

Entities:

- STUDENT
- COURSE

Relationship:

A student takes courses.

In an academic environment, a student can enroll in multiple courses during a semester, and a course can be taken by multiple students. Because both sides allow multiple associations, the relationship between STUDENT and COURSE is many-to-many (M:N). This relationship cannot be implemented directly in a relational database, so an intersection entity, such as ENROLLMENT, is required. The ENROLLMENT entity represents the relationship and can store additional information related to the enrollment process.

There are three main types of cardinality used in relational data modeling.

- **One-to-One (1:1)**

In a one-to-one relationship, each record in one entity is associated with only one record in another entity, and vice versa. This type of relationship is often used to separate sensitive or optional data into different tables.



Figure 4. Example of One-to-One Relationship



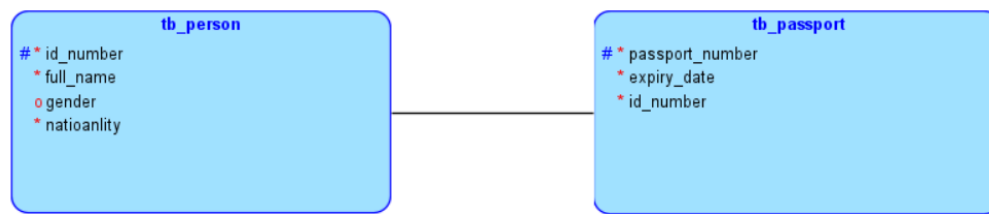


Figure 5. Example of One-to-One Relationship

- **One-to-Many (1:N)**

In a one-to-many relationship, one record in an entity can be associated with multiple records in another entity. This is the most common type of relationship in relational databases.

In implementation, the foreign key is placed on the table that represents the “many” side of the relationship.

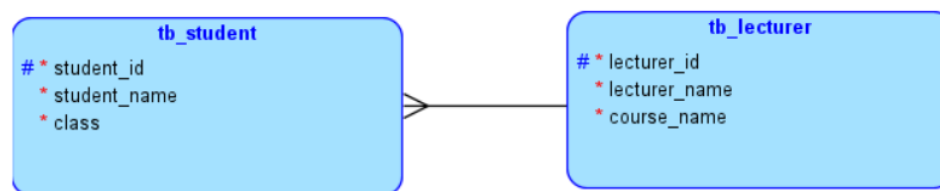


Figure 6. Example of One-to-Many Relationship

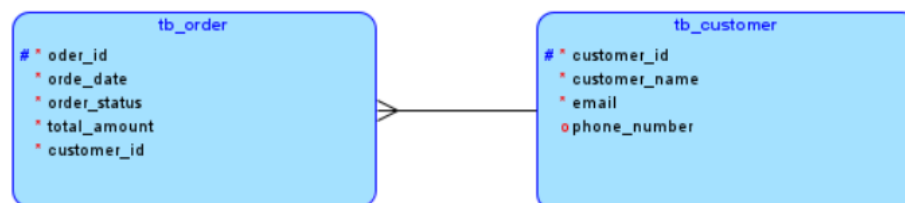


Figure 7. Example of One-to-Many Relationship



- **Many-to-Many (M:N)**

In a many-to-many relationship, multiple records in one entity can be associated with multiple records in another entity. This type of relationship cannot be implemented directly in a relational database.

To resolve this, an intersection entity is created. This entity breaks the many-to-many relationship into two one-to-many relationships and often contains additional attributes related to the relationship itself.

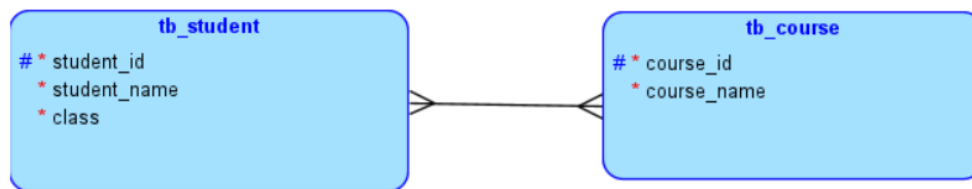


Figure 8. Example of Many-to-Many Relationship

In image 9, you will often find a Many-to-Many (M:N) relationship, such as a Student taking many Courses and a Course containing many Students. However, since a physical database cannot handle this direct connection without errors, we must fix it by adding a "middleman" table (Intersection Entity) like **study_result**. As shown in the figure, this connecting table sits in the middle and borrows the unique IDs from both the Student and Course tables to create a combined key, effectively breaking the complex M:N relationship into two tidy One-to-Many (1:N) relationships that the system can understand.

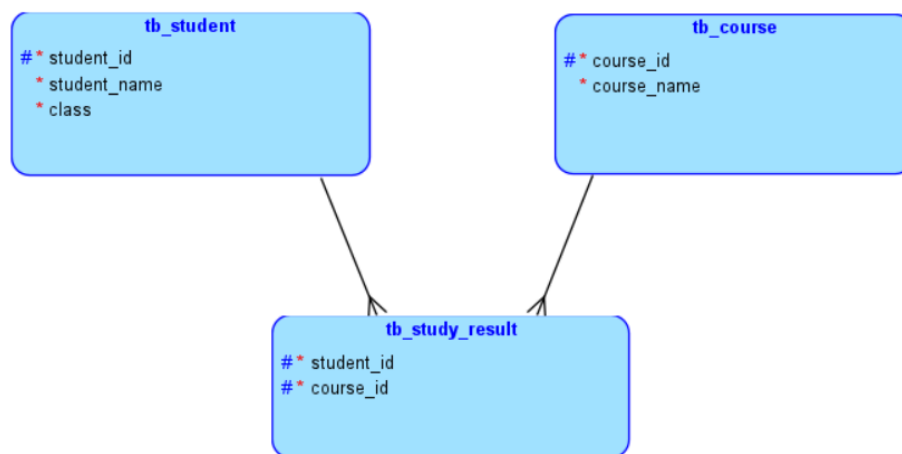


Figure 9. Example of study_result Connection Relationship



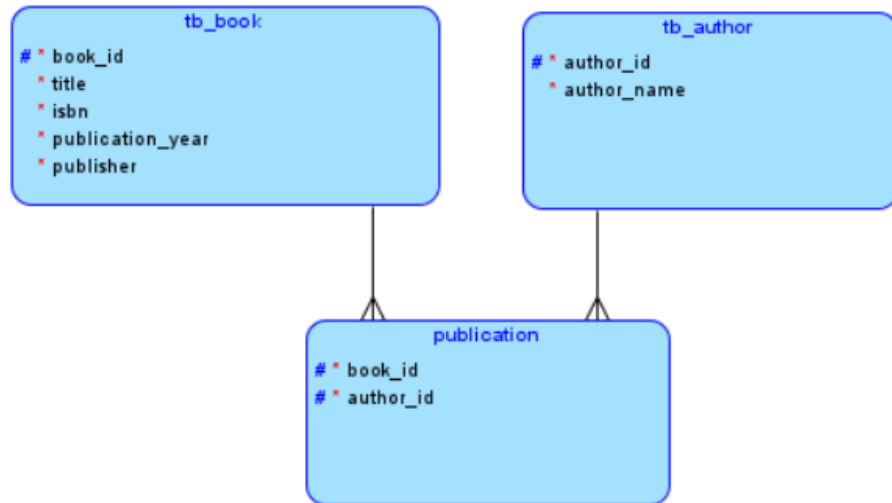


Figure 10. Example of publication Connection Relationship

You can also identify the rules of a relationship just by looking at the line styles, using the example of a Student and courses. The symbols at the ends of the line indicate "how many" (Cardinality): a single line means "One" (a Student belongs to one Course), while the three-pronged symbol means "Many" (a Student contains many Courses). Additionally, the line itself tells you if the connection is required (Optionality): a Solid Line means it is Mandatory (a Student *must* belong to a Course), whereas a Dashed Line means it is Optional.

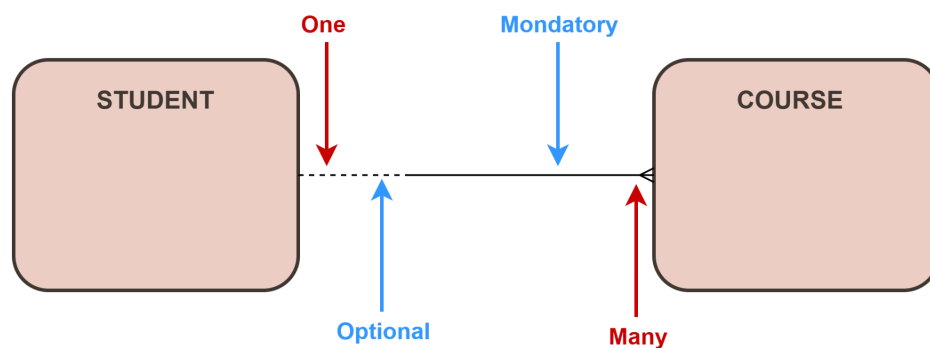


Figure 11. Example of Cardinality dan Optionality



OPTIONALITY

Optionality indicates whether an attribute or relationship must have a value. Optionality helps represent business rules and real-world constraints in database design.

There are two types of optionality:

1. Mandatory (*), meaning the value must exist.
2. Optional (o), meaning the value may be null.

Optionality affects data integrity and determines whether a record can exist without a related record in another table.

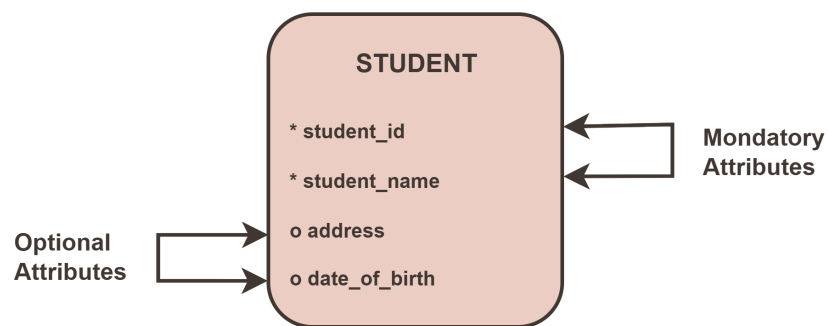


Figure 12. Example of Optional and Mandatory Attributes

3. PRECISION AND SCALE

Do you still remember precision and scale?

In the previous module, precision and scale were introduced as concepts used to control numeric values in a database. Precision defines the total number of digits that can be stored, while scale defines how many of those digits appear after the decimal point.

In this module, the focus shifts from *what precision and scale are* to how they are used in practice. You will learn how precision and scale are applied when defining numeric attributes in a data model, specifically using Oracle Data Modeler, and how these definitions affect the structure of a real database. Understanding the correct use of precision and scale is important to ensure data accuracy, consistency, and suitability for real-world scenarios, especially when working with numeric data such as quantities, prices, and measurements.



HOW TO USE PRECISION AND SCALE

In Oracle Database, numeric data is commonly defined using the NUMBER data type. The syntax for defining numeric values with precision and scale is:

NUMBER(p, s)

Where:

- **p (precision)** is the total number of digits that can be stored.
- **s (scale)** is the number of digits allowed after the decimal point.

This definition controls how numeric data is stored, validated, and restricted in the database. Choosing the correct precision and scale is part of good database design, not just a technical detail.

In Oracle Data Modeler, precision and scale are defined at the attribute (column) level when selecting the NUMBER data type. These values determine how the database will handle numeric input during real usage.

EXAMPLE 1 (CORRECT USAGE)

Attribute: price

price NUMBER(8, 2)

Explanation:

- The value can contain up to 8 digits in total.
- 2 digits are reserved for decimal values.
- Suitable for prices such as:
 - 125000.50
 - 999999.99

This definition is appropriate for monetary values where decimal precision is required.



EXAMPLE 2 (CORRECT USAGE)

Attribute: quantity

```
quantity NUMBER(5, 0)
```

Explanation:

- The value can contain up to 5 digits.
- No decimal values are allowed.
- Suitable for quantities such as:
 - 1
 - 250
 - 12000

This definition ensures that quantity values remain whole numbers.

EXAMPLE 3 (INCORRECT USAGE)

Attribute: total_score

```
total_score NUMBER(2, 2)
```

Why this is incorrect:

- Precision is 2, meaning only 2 digits are allowed in total.
- Scale is also 2, meaning both digits must be after the decimal point.
- This allows values only between:
 - 0.00 and 0.99

This definition is not suitable for a score that may exceed 1.

EXAMPLE 4 (CORRECT USAGE)

Attribute: total_score

```
total_score NUMBER(5, 2)
```

Why this is correct:

- Allows up to 5 digits in total.
- Allows 2 digits after the decimal point.
- Suitable for values such as:
 - 75.50
 - 100.00



EXAMPLE 5 (INCORRECT USAGE)

Attribute: age

```
age NUMBER(10, 2)
```

Why this is incorrect:

- Age does not require decimal values.
- Scale of 2 allows unnecessary decimal precision.
- This may cause confusion and improper data input.

EXAMPLE 6 (CORRECT USAGE)

Attribute: age

```
age NUMBER(3, 0)
```

Why this is correct:

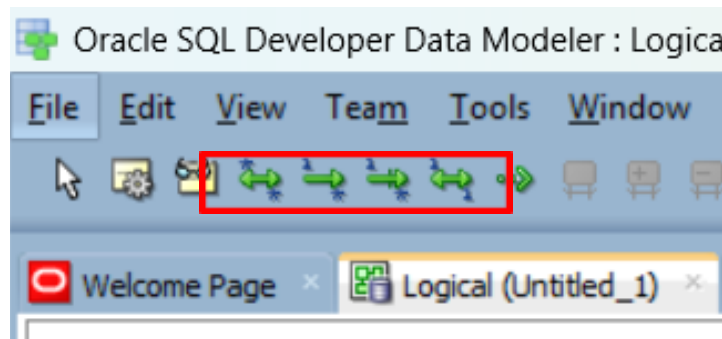
- Age is stored as a whole number.
- Precision and scale match real-world data behavior.
- Data becomes easier to validate and understand.



PRACTICE ACTIVITY

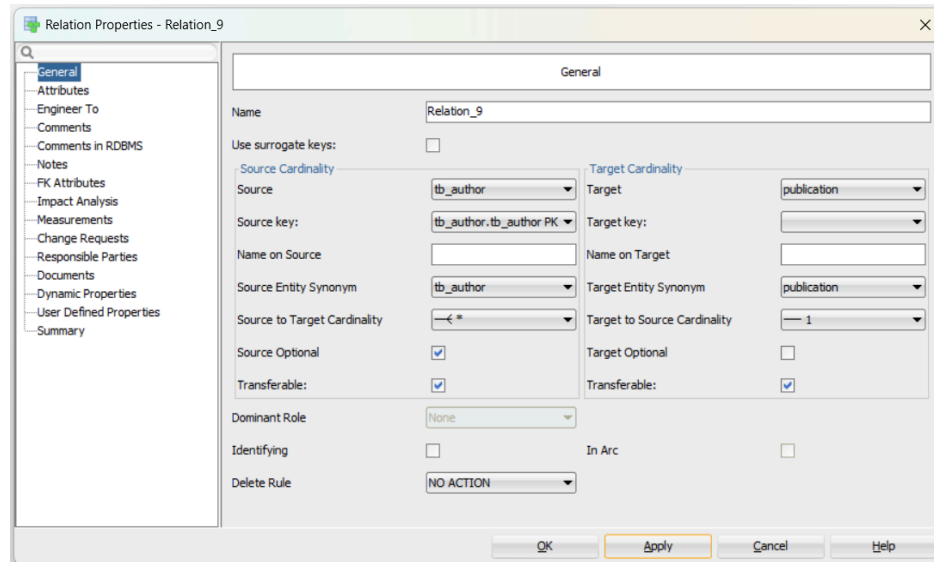
TUTORIAL FOR ADDING RELATIONSHIPS TO A CONCEPTUAL DATA MODEL (CDM)

1. Ensure that at least two (2) entities are available in the diagram, in this case **tb_book** and **tb_author**.
2. Determine the type of relationship to be created.
For this example, the relationship between **tb_book** and **tb_author** is **Many-to-Many (M:N)**.
3. Because a many-to-many relationship cannot be implemented directly, prepare an intersection entity named publication.
4. Select the appropriate relationship tool from the relationship menu provided in Oracle Data Modeler (as indicated by the red box in the figure).



5. Create the first relationship by drawing a line from **tb_book** to publication.
After drawing the line, the "Relationship Properties" window will appear automatically.
6. In the "General" tab of the "Relationship Properties" window:
 - Set the relationship name, and
 - Define the cardinality between **tb_book** and publication according to the design requirements.
7. Create the second relationship by drawing a line from **tb_author** to publication, then open the "Relationship Properties" window again.
8. In the "General" tab, configure the relationship as follows:
 - Set the appropriate relationship name,
 - Define the cardinality between **tb_author** and publication.





9. Observe the red and green boxes in the figure:

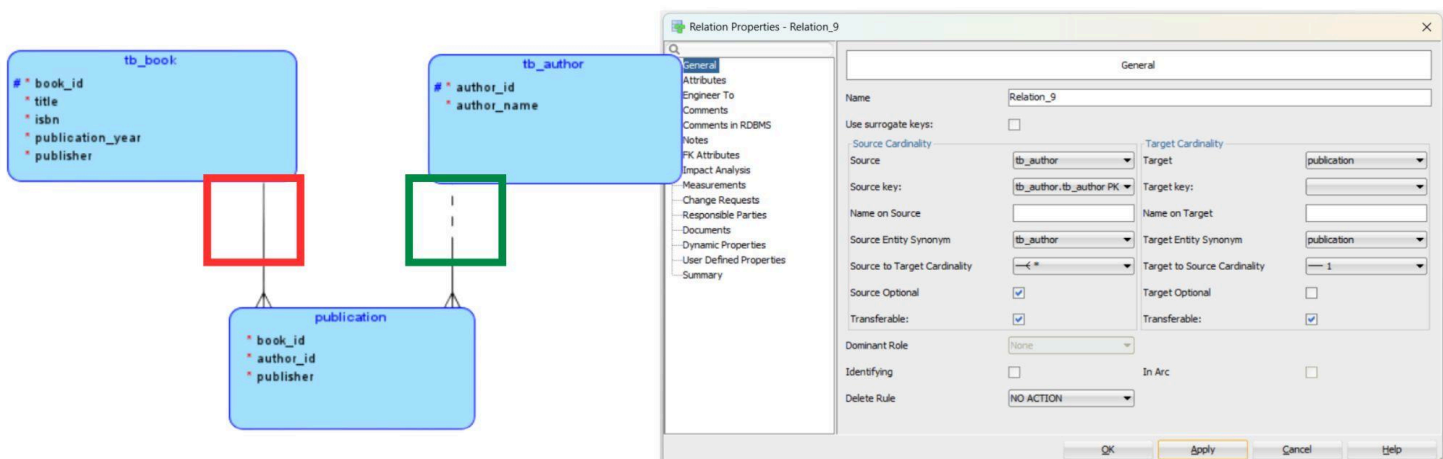
- The red box represents the cardinality line connected to the entity.
- The green box represents the optionality symbol of the entity.

10. For the relationship between **tb_author** and publication, enable the “Source Optional” option to indicate that the relationship is **optional** on the **tb_author** side.

As a result, the relationship line will appear as a **dashed line**.

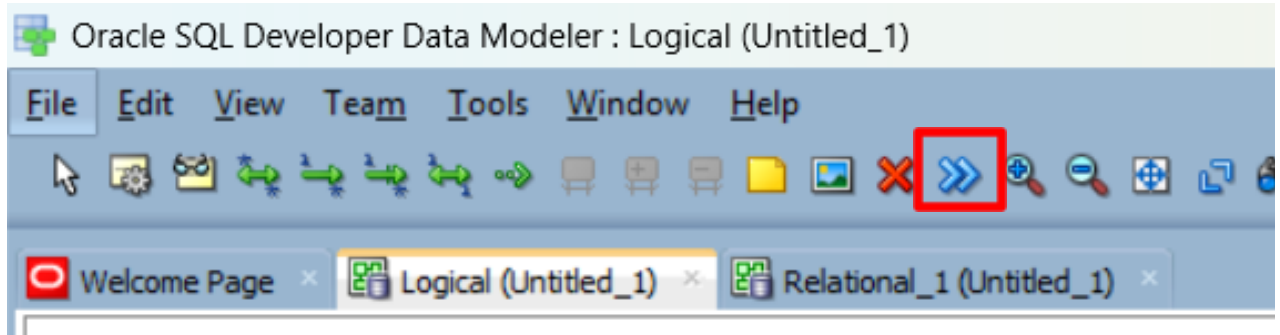
11. For the publication entity, do not enable the optional option.

This indicates that the relationship is **mandatory** for publication, and the relationship line will appear as a **solid line**.

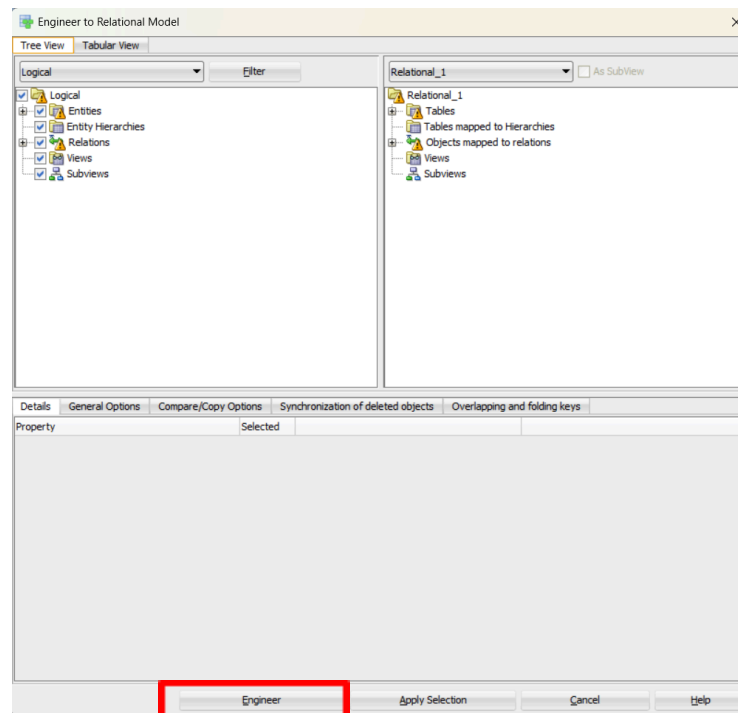


TUTORIAL ON CONVERTING A CONCEPTUAL DATA MODEL (CDM) TO A PHYSICAL DATA MODEL (PDM)

1. After all entities and relationships are formed in the CDM, choose the icon in the red box below to convert it.



2. Press the "Engineer" button on the window that appears.



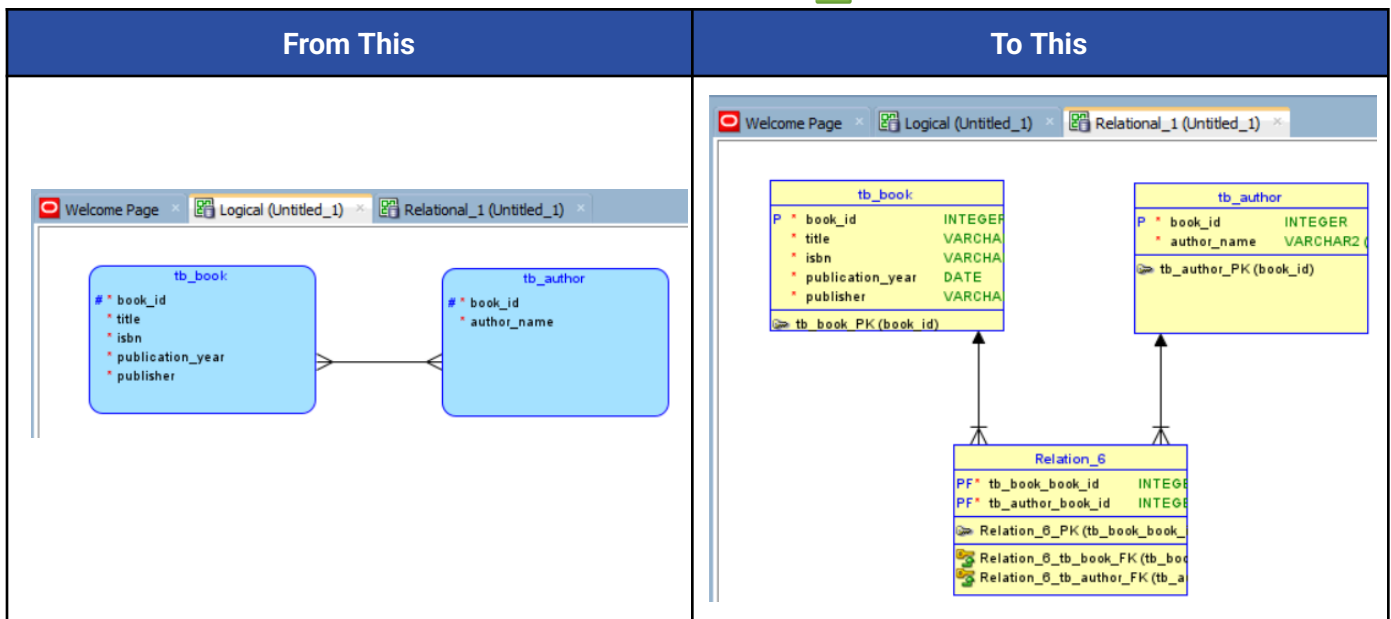
3. Pressing that button will lead you to the "Relational_1 (Untitled_1)" worksheet, with the result shown in the figure.



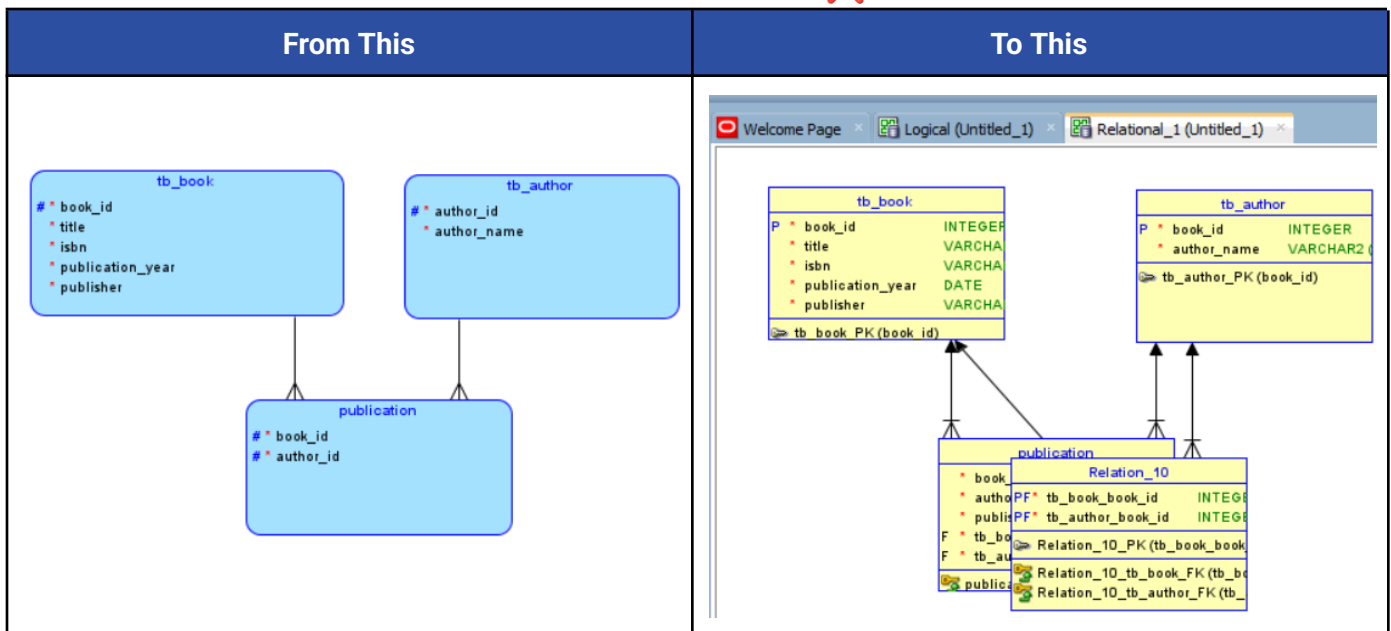
THINGS TO CONSIDER WHEN CONVERTING A CONCEPTUAL DATA MODEL (CDM) TO A PHYSICAL DATA MODEL (PDM)

1. For entities with a Many-to-Many (M:N) relationship, you are not required to manually create the intersection entity, as it will be automatically created during the conversion to the PDM.

Correct Conversion

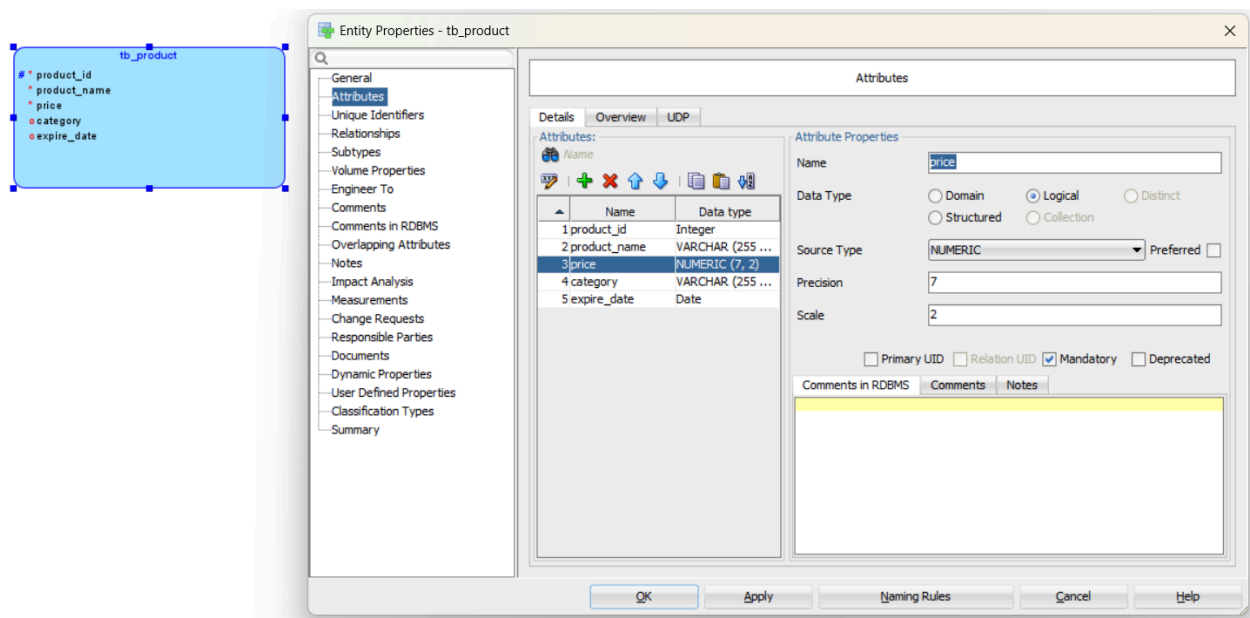


Incorrect Conversion



TUTORIAL FOR APPLYING PRECISION AND SCALE TO AN ATTRIBUTE

1. First, prepare one entity and add one attribute with the NUMERIC data type. Set the data type in the "Source Type" field.
2. Next, configure the precision and scale according to your requirements. In the example shown, precision 7 and scale 2 are used.
3. Click "Apply" and then "OK".
4. The "price" attribute with the data type NUMERIC(7, 2) has now been successfully created.



TRY OUT

In this Try Out section, you are required to extend the existing data model by adding relationships between entities that have been created in the **previous module**, using an **Online Shop** theme.

You must work with the existing entities, complete with attributes, to form meaningful relationships within the model.



TRY OUT INSTRUCTIONS

1. Use the existing entities provided in the previous module as part of your model.
2. Add one numeric attribute to one of the existing entities. The numeric attribute must apply precision and scale using the NUMBER(p, s) concept, and the chosen values must be appropriate for the data.
3. Define and create relationships between:
 - tb_product and the newly added entities, and/or
 - the newly added entities themselves, if required by your design.
4. For each relationship:
 - determine the relationship type (One-to-One, One-to-Many, or Many-to-Many),
 - define the appropriate cardinality and optionality.
5. Ensure that the relationships you create are logical and consistent with real-world Online Shop rules.

NB:

1. **You are allowed to add additional entities if required by your design.**
2. **Any additional entity must be relevant to the Online Shop theme and properly designed with attributes.**

ASSIGNMENT

Create and extend a Conceptual Data Model (CDM) individually by continuing the schema you have designed in the previous module.

[CHECK_YOUR_SCHEMA_HERE](#)

ASSIGNMENT REQUIREMENTS

In the CDM schema that has been created with individual schema, add new attributes to only 3 entities. Provide mandatory (*) or optional (o) markings for these attributes.

1. Use the entities created in the previous module and add at least two (2) relationships between entities.
2. For each relationship, clearly define:
 - relationship type (One-to-One, One-to-Many, or Many-to-Many),
 - cardinality, and
 - optionality.
3. Ensure that each entity involved in the relationship has a valid Unique Identifier (UID), whether it is simple or composite.



4. Add at least one (1) new numeric attribute to an existing entity that applies precision and scale, using the `NUMBER(p, s)` concept. The chosen precision and scale must be appropriate for the nature of the data.
5. If none of the existing entities are suitable for adding a numeric attribute with precision and scale, you are allowed to add a new entity. Any new entity must:
 - be relevant to the schema,
 - have appropriate attributes, and
 - include at least one numeric attribute that applies precision and scale.

NB:

1. You are allowed to add additional entities if required by your design.
2. Any additional entity must be relevant to your schema and properly designed with attributes.

ASSESSMENT

You must explain this assessment to the assistant during the demo week.

1. Choose one entity from your CDM. Mention the type of UID used in that entity and explain why it is suitable.
2. Choose one relationship from your CDM. Explain the cardinality of the relationship based on how the data works in the real world.
3. Using the same relationship, explain the reason for assigning optional and mandatory relationships.
4. Identify one numeric attribute in your CDM that uses precision and scale. Explain why the selected precision and scale values are appropriate for that attribute.
5. Explain why defining relationships and numeric attributes is important when designing a CDM.



PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Try Out	Total 5%
- At least two (2) relationships between entities are created. - Relationship types (1:1, 1:N, or M:N) are correctly defined. - Cardinality is defined on both sides of each relationship. - Optionality (mandatory / optional) is defined correctly on both sides. - One numeric attribute applies precision and scale (NUMBER(p, s)). - The chosen precision and scale values are appropriate for the data.	100
- Two (2) relationships are created. - Relationship types are correct. - Cardinality is defined on both sides, but optionality is missing on one side, or - Numeric attribute with precision and scale exists, but precision or scale value is not appropriate.	80
- Two (2) relationships are created, but one relationship type is incorrect, or - Cardinality is defined on only one side of a relationship, or - Numeric attribute exists but precision and scale are not defined correctly.	70
- Only one (1) relationship is created, or - Relationships are created but cardinality and optionality are not defined, or - Numeric attribute is added but does not apply precision and scale.	60
- A relationship is attempted but relationship type, cardinality, and optionality are not defined, or - Numeric attribute is added without applying NUMBER(p, s).	40
- No relationship is created, or - No numeric attribute is added, or - The Try Out instructions are not followed.	0
Assignment	Total 30%
- At least two (2) relationships are created. - Relationship types, cardinality, and optionality are defined correctly for all relationships. - All entities involved in relationships have valid UUIDs. - One numeric attribute applies precision and scale (NUMBER(p, s)) with appropriate values.	100



The CDM is consistent with the previous module and ready for further development.	
- Two (2) relationships are created. - Relationship types and cardinality are defined correctly. - Optionality is missing on one side of one relationship. - All entities have UIDs. - Numeric attributes with precision and scale exist, but precision or scale value is not appropriate.	80
- Two (2) relationships are created. - One relationship has an incorrect type, OR - Cardinality or optionality is missing in one relationship. - Some entities involved in relationships have unclear or incorrect UIDs. - Numeric attributes exist but precision and scale are not defined correctly.	70
- Only one (1) relationship is created, OR - Relationships are created but cardinality and optionality are not defined, OR - Numeric attribute is added but does not apply precision and scale.	60
- A relationship is attempted but relationship type, cardinality, and optionality are not defined, OR - Numeric attribute is missing.	40
- No relationship is created, OR - No numeric attribute is added, OR - The assignment instructions are not followed.	0
Assessment	Total 65%
- Correctly explains the UID type used in one chosen entity and explains why it is suitable. - Correctly explains the cardinality of one relationship based on real-world data behavior. - Correctly explains the reason for assigning mandatory and optional relationships using the same relationship. - Correctly explains one numeric attribute that applies precision and scale (NUMBER(p, s)), including why the selected values are appropriate. - Clearly explains why relationships and numeric attributes are important in CDM design. - All explanations are consistent with the CDM.	100



• Correctly explains the UID type, but the explanation of suitability is incomplete. • Correctly identifies the relationship cardinality, but the real-world explanation is incomplete. • Correctly identifies mandatory and optional relationships, but the reason is incomplete. • Numeric attributes with precision and scale are identified, but the explanation of value selection is incomplete. • Explains the importance of relationships and numeric attributes, but only at a general level.	80
• Can mention the UID type but cannot clearly explain why it is suitable. • Can mention relationship cardinality but cannot relate it clearly to real-world rules. • Can mention mandatory and optional relationships but cannot explain the reason. • Numeric attributes are mentioned, but precision and scale are not explained correctly. • Explanation of the importance of relationships and numeric attributes is very general.	70
• Cannot correctly explain the UID type used in the CDM. • Cannot correctly explain relationship cardinality. • Mandatory and optional relationships are incorrectly explained. • Precision and scale are not understood or incorrectly explained. • Doesn't understand why relationships and numeric attributes are important in CDM design.	60
• Can answer only one of the assessment questions correctly. • Most explanations are not based on the CDM.	40
• Does not answer the assessment questions, or • Answers are not related to the CDM, or • The student is absent during the assessment.	0

NB:

1. If a student attends the material session and receives a score for the practicum activities, but is unable to fully answer the assessment questions during the demo week, the student must still be given a minimum assessment score of 40.
2. If a student does not attend the material session, does not complete the Try Out, receives a score of 0 for the Assignment, and cannot answer the assessment questions at all, then the assessment score must be 0.

